KENNESAW STATE
UNIVERSITY

CS 4277
Graph Neural Network

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

College of Computing and Software Engineering

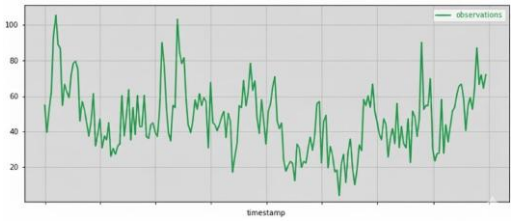
Content

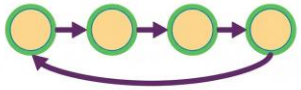
- Introduction to GNN and Challenges
- Graph basics
- Graph neural networks
- Over smooth and graph attention
- Ecosystems for GNNs
- Advanced topics

Introduction

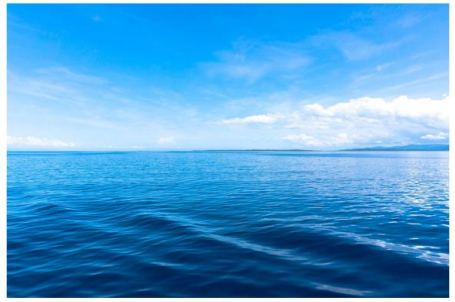
Grid-structured data

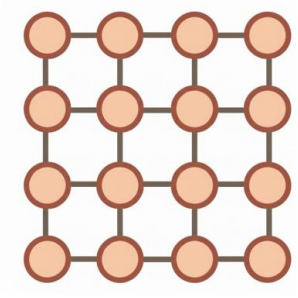
- Sequential Data: Recurrent Neural Network



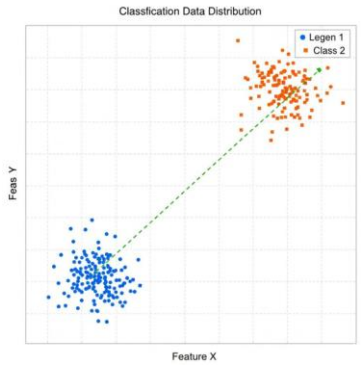


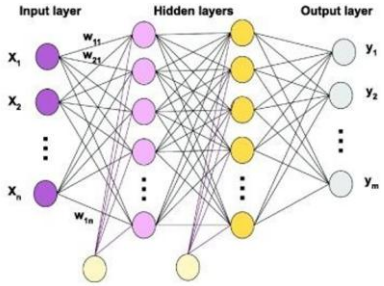
- Images Convolutional Neural Network





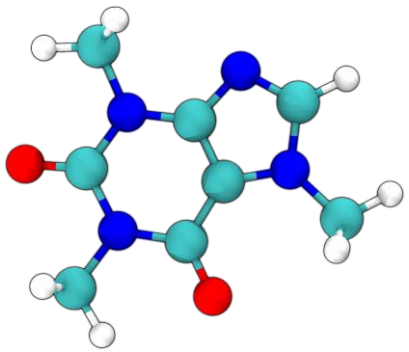
- Tabular Data: MLP



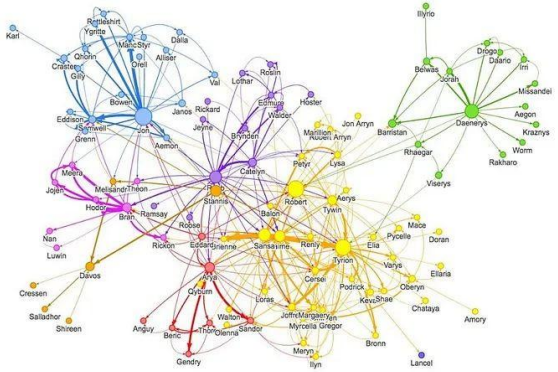


How about

- 3d representation of the Caffeine molecule



- Social Network

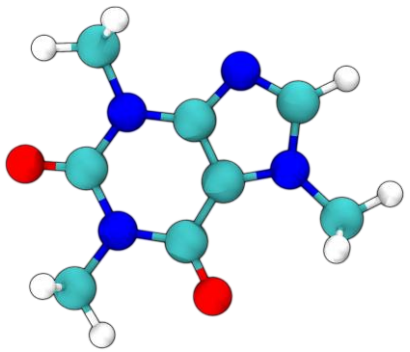


Why graphs matter?

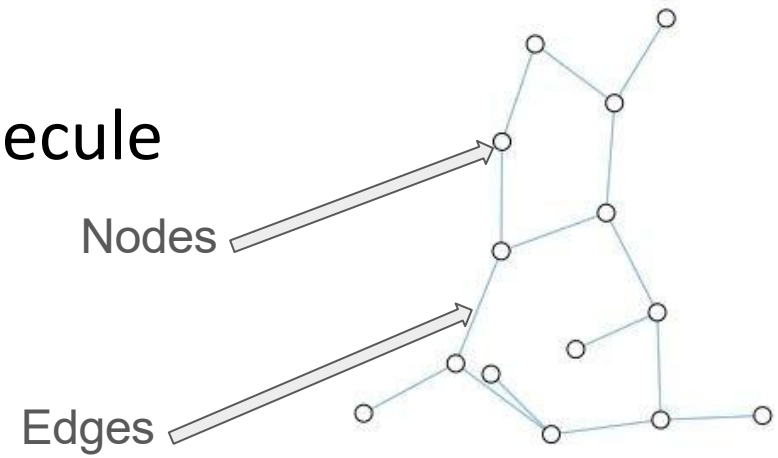
- Social networks: Relationships, influence propagation
- Molecules: Chemical properties depend on structure
- Citation networks: Knowledge graphs, recommendations
- Traffic systems: Routing, congestion prediction

Why graphs matter?

- 3d representation of the Caffeine molecule

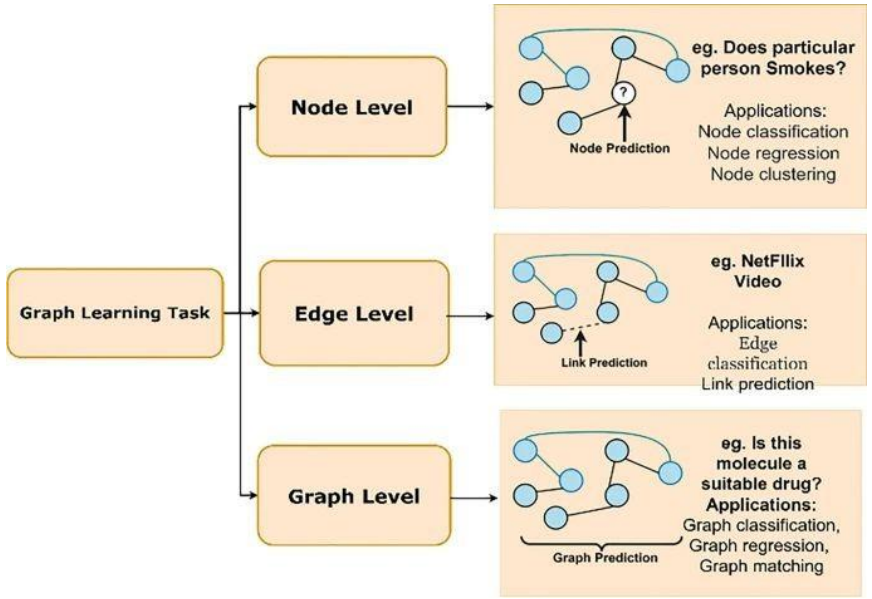


- Graph representation of the Caffeiine molecule



Types of Graph Neural Networks Tasks

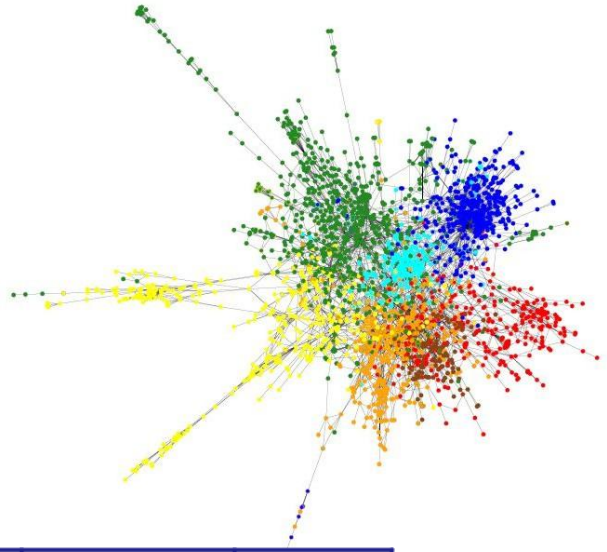
- Node Classification
  - Classify each node in the graph
  - Graph shows connections between each data point
  - No pooling
  - GCNNs vs. MLPs
- Graph Signal Classification
  - Classify the entire graph signal
  - Graph shows connections between the features
  - Graph can be different for each data point
  - Can do pooling



Node Classification: Citation Network

- Classify the type of paper using the citation network (**graph**) and a bag of words (**data**) for each paper (**node**)

Color represents classes

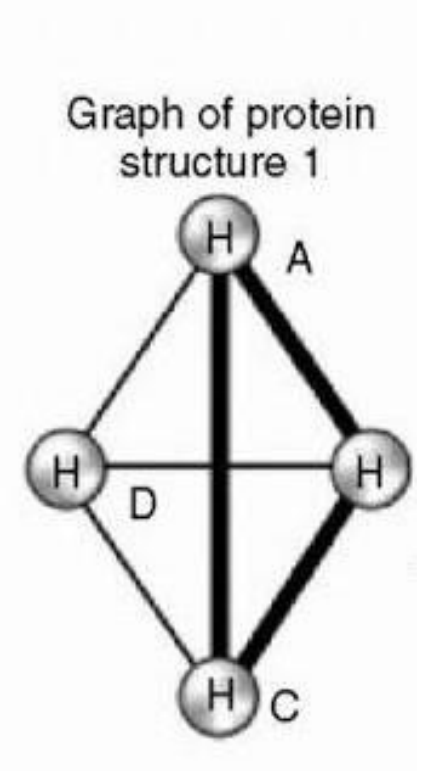


Dataset	Nodes	Labelled Nodes	Label Rate	Edges	Connectivity	Features	Classes
Citeseer	3327	120	0.036	4732	4e-4	3703	6
Cora	2708	141	0.052	5429	7e-4	1433	7
Pubmed	**19717**	**59**	0.003	44338	**1e-4**	500	3

Graph Classification: Biological Network

- Each protein, chemical and enzyme structure is a different graph
- Data is the elements of each node in the structure
- Predict properties based on the chemical structure

Dataset	#Graphs	Classes	Avg. #nodes	Avg. # edges
MUTAG	188	2	17.7	38.9
PTC	344	2	26.7	50.7
ENZYME	600	6	32.6	124.3
D&D	1178	2	284.4	1921.6



Graph Classification: More Examples

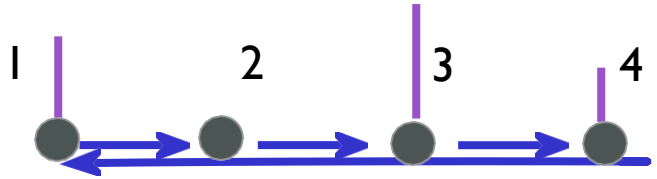
- IMDB-binary:
  - ego-networks of actors who have appeared together in movies. classify network into action/romance movie
- Reddit-binary:
  - does the user come from Q&A forum or discussion forum?

Why should we Deep Learning people care?

- Real-World Applications
  - Drug Discovery: Predict molecular properties, design better compounds
  - Recommendation: User-item interactions, collaborative filtering
  - Forecasting: Traffic, weather, energy consumption on grids

Challenges in Graph Data

- Discrete Signal Processing (DSP) works well on time and images



- But fail for graph data
  - No fixed ordering
  - No way to incorporate graph data.

Challenges in handling Graph Data

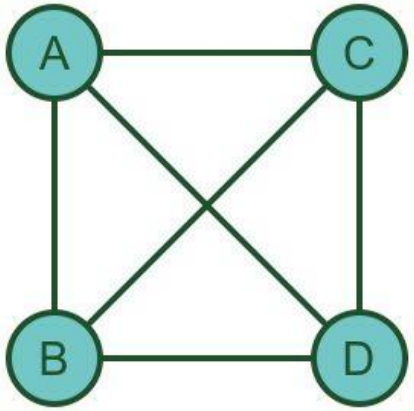
- Models like MLPs, CNNs, and RNNs are built on the assumption that data is structured in a regular, fixed grid (like pixels in an image or steps in a sequence).

- Problem:
  - Graphs are irregular and unstructured. Nodes can have a variable number of neighbors, and the connections don't follow a simple, predetermined pattern.
  - Graphs lack a natural order. Models must be permutation-invariant because re-labeling nodes cannot change the result.
  - Standard models cannot easily incorporate the connectivity information. They need a mechanism to aggregate features from neighbors

Graph Basics

Graph Basics

- Graph is an ordered pair G = (V, E):
  - Nodes V (set of vertices)
  - Edges E (associated with two distinct vertices) connecting them



Complete graph

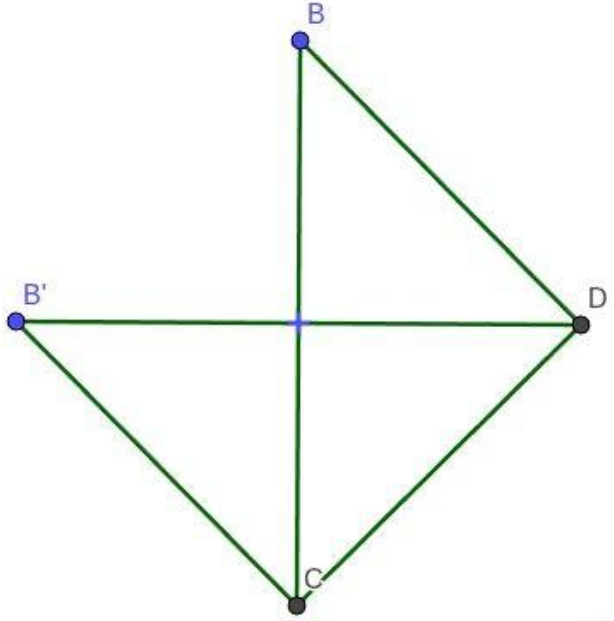
	A	B	C	D
A	0	1	1	1
B	1	0	1	1
C	1	1	0	1
D	1	1	1	0

Adjacency Matrix

- It is a n × n matrix in which:
  - n is the number of vertices
  - A(i, j) = 1 only if there is a link from i to j and
  - A(i, j) = 0 if not

Adjacency Matrix:

$$\begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$



Other common representation

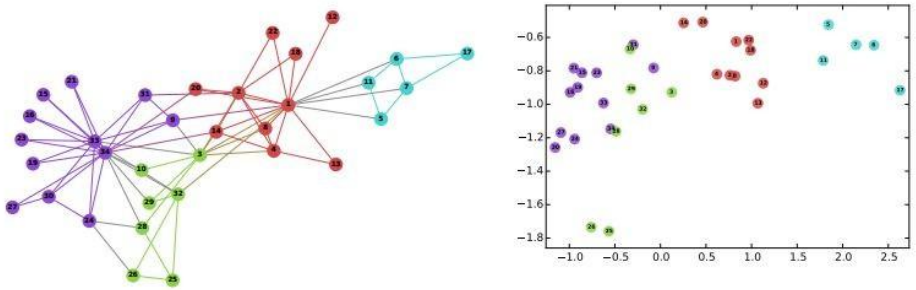
- Edge List
  - An edge list is a collection of pairs (or tuples), where each pair (u, v) indicates that there is an edge connecting node u to node v
- Adjacency List
  - It is implemented as an array or map where the index/key represents a vertex, and the value is a list of all vertices adjacent to it (its neighbors).

Embeddings

- Transform complex, high-dimensional data (like words or images) into dense, low-dimensional numerical vectors that machine learning models can easily process
- Word Embeddings:
  - Convert words into vectors such that words with similar semantic meaning or that appear in similar contexts are positioned close together in the vector space
  - Example: GloVe, Word2Vec, BERT
- Image Embeddings:
  - Compress the high-dimensional pixel data into a dense vector that encodes the image's visual features (objects, patterns, textures)
  - Example: CNN, Vision Transformer
- These work well for unstructured or sequential data. Fail to capture the defining characteristic of graph data: interconnectedness and complex relationships.

Graph Node Embeddings

- To encode the topology and structural properties of the network. This includes:
  - node's own features
  - Its local neighborhood (dense group of nodes within a larger graph structure)
  - Its structural role
- Adjacency Matrix are often huge and sparse
- Graph embeddings condense this high-dimensional, sparse data into a low-dimensional, dense vector space, such that similar nodes in the graph are embedded close together.



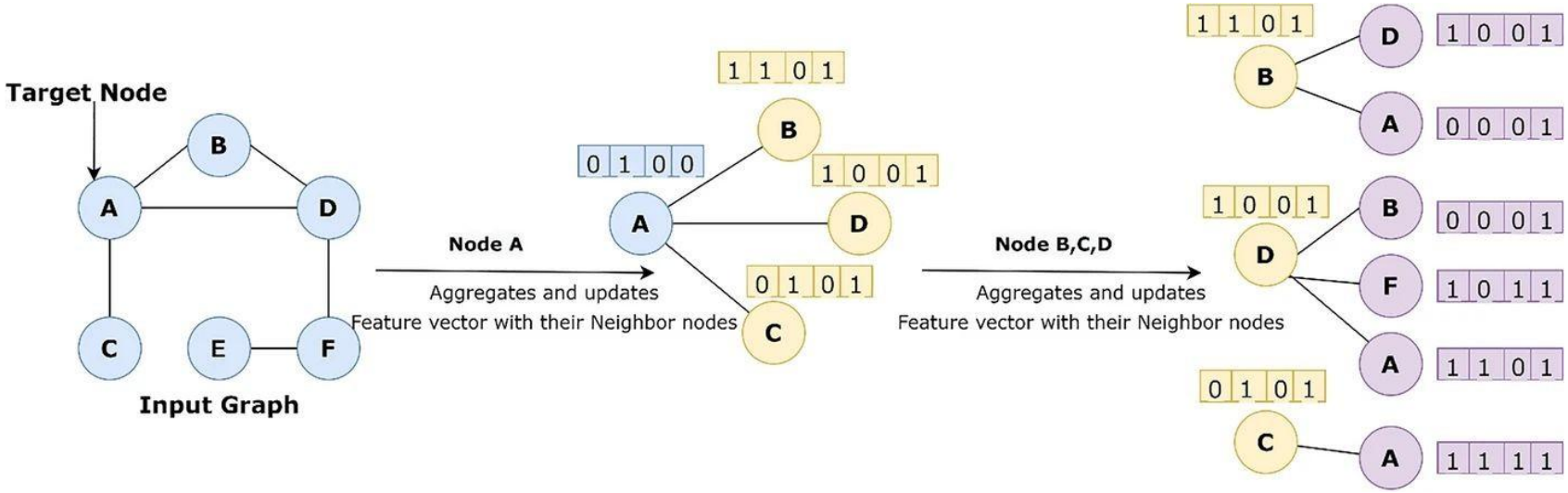
(a) Input: Karate Graph (b) Output: Representation

Graph Neural Networks

Graph neural networks (GNNs)

- Designed to handle irregular connectivity and the lack of inherent order for nodes.

- The core mechanism Message Passing
  - Gather
  - Aggregate
  - Update



Khemani, B., Patil, S., Kotecha, K. and Tanwar, S., 2024. A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions. *Journal of Big Data*, *11*(1), p.18.

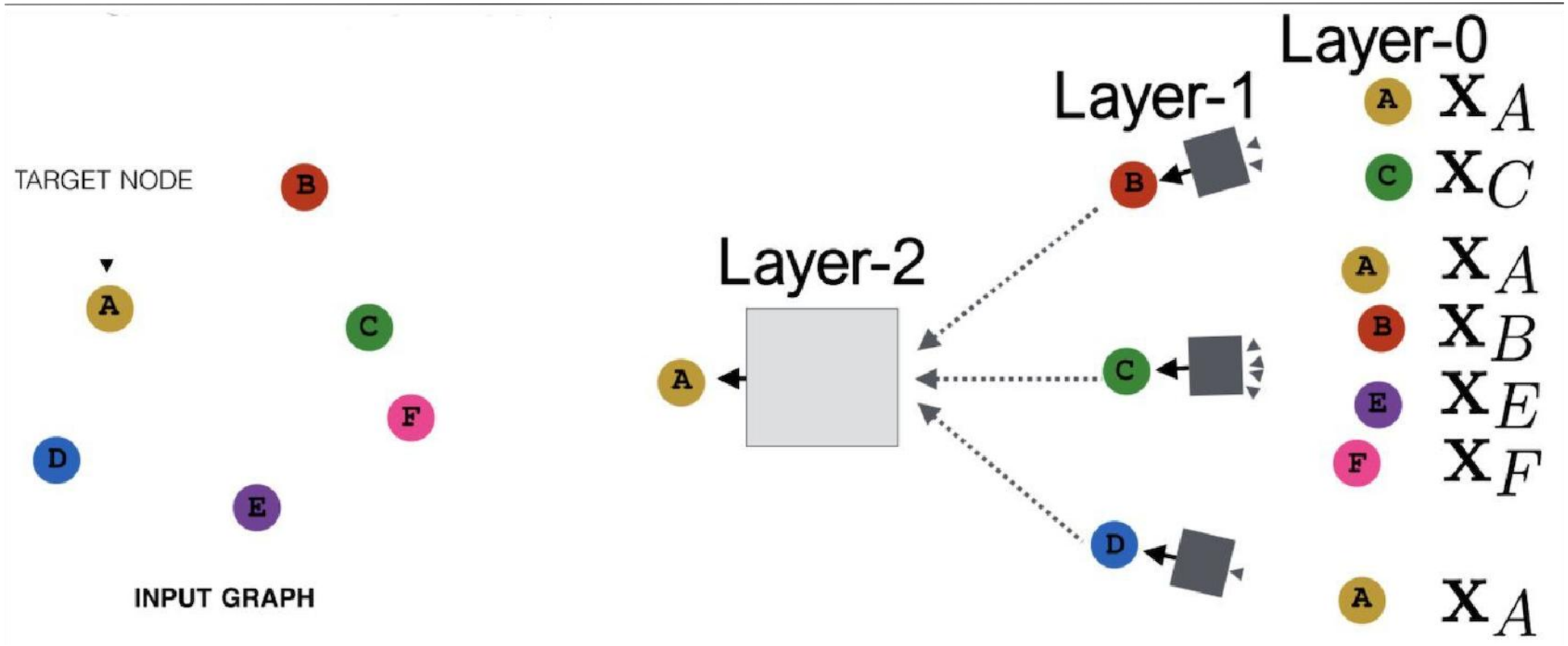
Graph Neural Networks (GNNs)

- The Neighbourhood N_{i} of a node i is defined as the set of nodes j connected to i by an edge
  - Gather: Each node will create a message, which will be sent to other nodes later
    - Example: A Linear layer $\boldsymbol{h}_{\boldsymbol{u}}^{(\boldsymbol{l})} = W^{(l)} h_u^{(l-1)}$
  - Aggregate: Node v will aggregate the messages from its neighbors $u \in N\ (v)$ and combines them into a single summary vector $\boldsymbol{h}_{\boldsymbol{v}}^{(\boldsymbol{l})}$

$$h_v^{(l)} = AGG\left(\left\{m_u^{(l)}, u \in N(v)\right\}\right)$$

  - Node Update: Node v combines the aggregated message $\boldsymbol{h}_{\boldsymbol{v}}^{(\boldsymbol{l})}$ with its own feature vector from the previous layer $\boldsymbol{h}_{\boldsymbol{v}}^{(\boldsymbol{l}-\mathbf{1})}$. Easier to include $\boldsymbol{h}_{\boldsymbol{v}}^{(\boldsymbol{l}-\mathbf{1})}$ when computing $\boldsymbol{h}_{\boldsymbol{v}}^{(\boldsymbol{l})}$.
- This process is repeated L times (where L is the number of GNN layers) to capture features from the L-hop neighborhood.

Graph Neural Networks (GNNs)



Graph Neural Networks (GNNs)

- $h_v^0 = x_v$, initial 0-th layer embeddings are equal to node features

$$h_v^{(k+1)} = \sigma\left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)}\right), \forall k \in \{0, \cdots, K-1\}$$

- σ: non-linearity, e.g. ReLU
- $h_v^{(k)}$: embedding of v at layer k
- $\sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|}$, average of neighbor's previous layer embeddings
- K total number of layers
- $z_v = h_v^{(K)}$, embedding after K layers of neighborhood aggregation

Aggregation Functions & Information Propogation

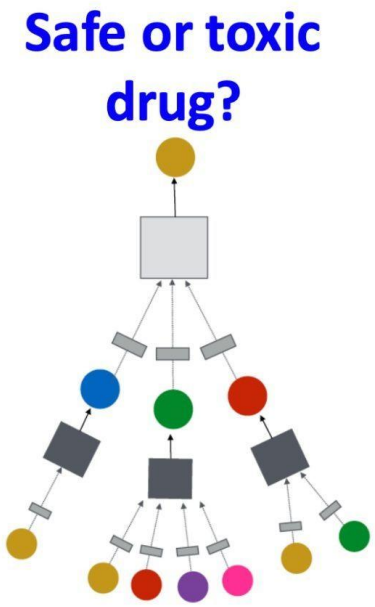
- Aggregation function must be permutation-invariant.
- Commonly used are - Sum, Mean/Average, Max Pooling & Attention
  - Sum Aggregation: $h = \Sigma m$ (most common, permutation invariant)
  - Mean Aggregation: $h = (1/|N|)\, \Sigma\, m$ (normalized version)
  - Max Aggregation: $h = \max(m)$ (captures outliers)
- Information Propagation

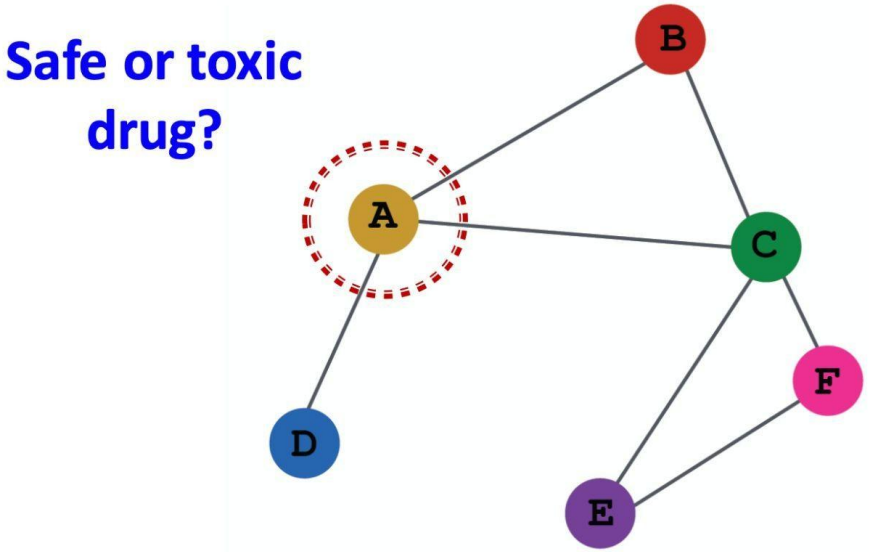
Layer 1	→	Layer 2	→	Layer L
Direct neighbors		2-hop neighbors		L-hop neighborhood

Typical: 2-4 layers (diminishing returns & over-smoothing)

GNN Model Training

- Directly train the model for a supervised task (e.g., node classification)





E.g., a drug-drug interaction network

GNN Model Training

- Use cross entropy loss for Node Classification

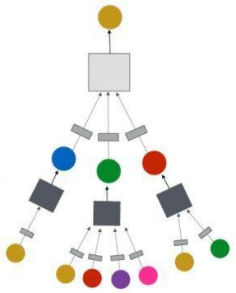
$$\mathcal{L} = -\sum_{v \in V} y_v \log(\sigma(z_v^T \theta)) + (1 - y_v)\log(1 - \sigma(z_v^T \theta))$$

Encoder output:
node embedding

Classification weights

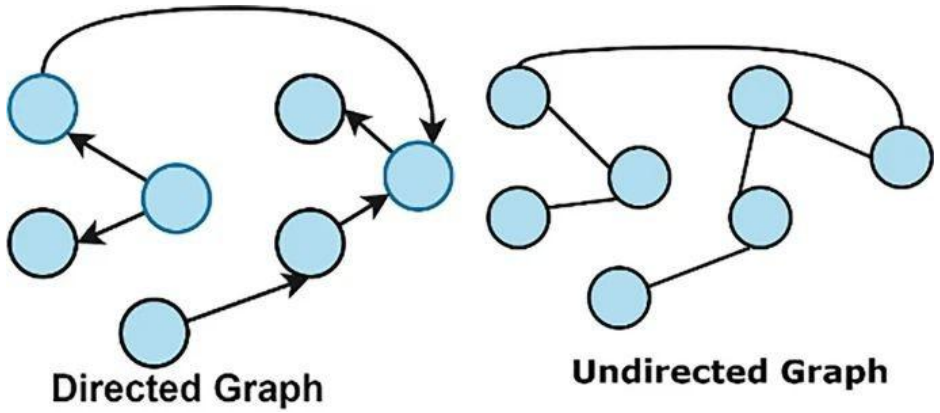
Node class label

Safe or toxic drug?



What Types of Graphs can we train?

- Directed/undirected graphs:
- Focus is more on the flow of information. Standard Graph Convolutional Networks (GCNs), GraphSAGE is typically implemented for **undirected** graphs
- For directed graphs a node v will only aggregate the messages from its neighbors $u \in N\ (v)$ (nodes u such that $u \rightarrow v$)



Are GNNs enough?

- The first GNN paper proposed a recurrent approach that required iteration to a fixed point.
- The Issue: Achieving convergence takes a significant, variable number of steps and requires backpropagation through a potentially very long sequence of updates.
- This made the original GNNs computationally expensive and slow to train, especially on large graphs.

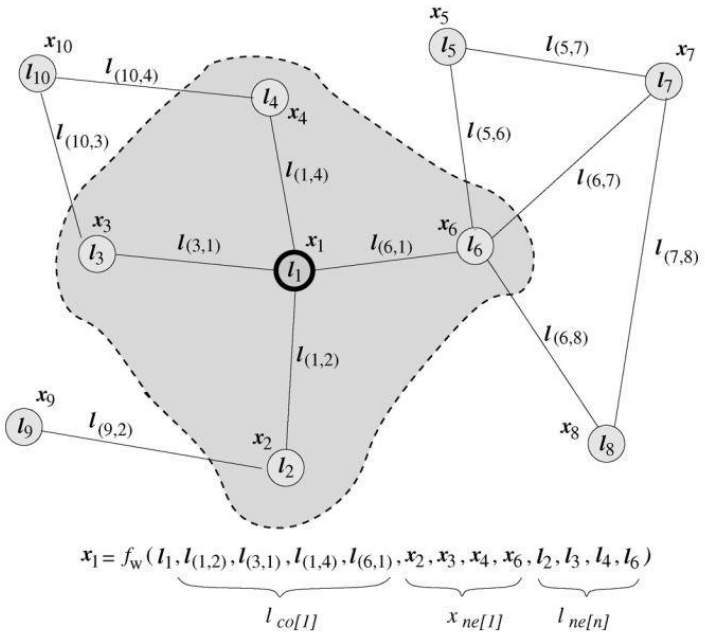
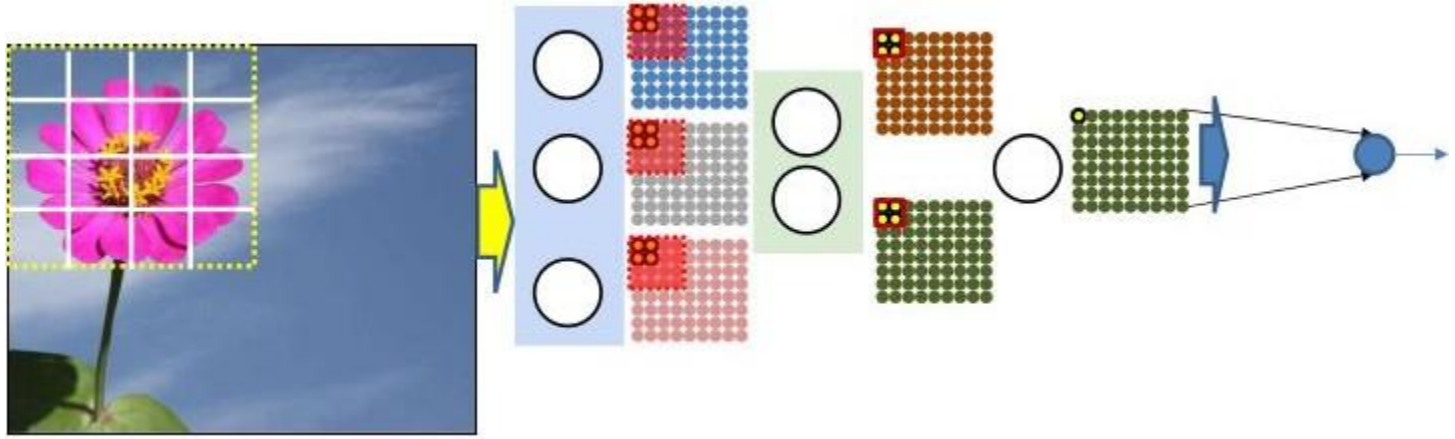


Fig. 2. Graph and the neighborhood of a node. The state x_1 of the node 1 depends on the information contained in its neighborhood.

Solution: Convolution

- What was the benefits of Convolution?
  - Extracts Local Patterns: Filters learn to recognize features by looking at a fixed, local neighborhood of pixels
  - Achieves Parameter Sharing: The same filter (weights) is applied across the entire image, making the model highly efficient and scalable
  - Is Translation Invariant: The model can recognize a feature regardless of where it appears in the image

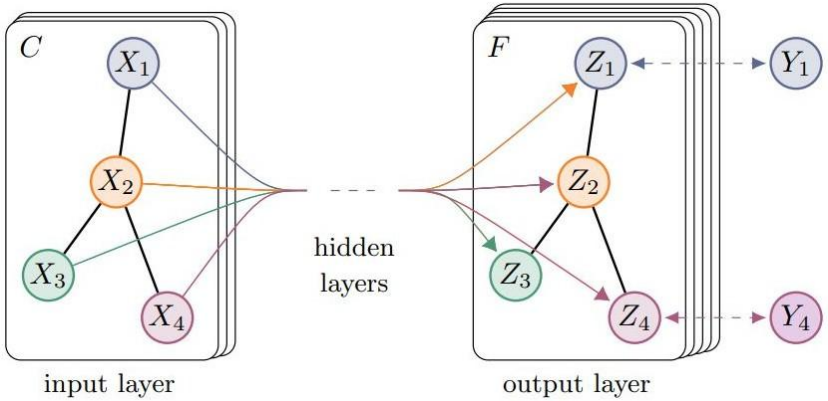


Graph Convolutional Networks (GCN)

- Foundational GNN architecture (Kipf & Welling 2017, Semi-supervised Classification with Graph Convolutional Networks)

- Update rule:

- $h = \sigma(W \times \Sigma\,(\hat{A}[i,j] * h))$
  - $\hat{A}$ = normalized adjacency matrix
  - W = learnable weight matrix
  - σ = ReLU activation



- This introduced scalable GCN model that uses the matrix multiplication formula to implement the message-passing (or convolution) step.
- Solves convergence and complexity issues. Pushed GNNs into the mainstream.

Kipf et al. SEMI-SUPERVISED CLASSIFICATION WITH GRAPH CONVOLUTIONAL NETWORKS

GNN vs GCN

- GNN:

$$h_i^{(l+1)} = W_1 h_i^{(l)} + W_2 \sum_{j \in N(i)} e_{j,i} x_j^{(l)}$$

- GCN: vectorized representation

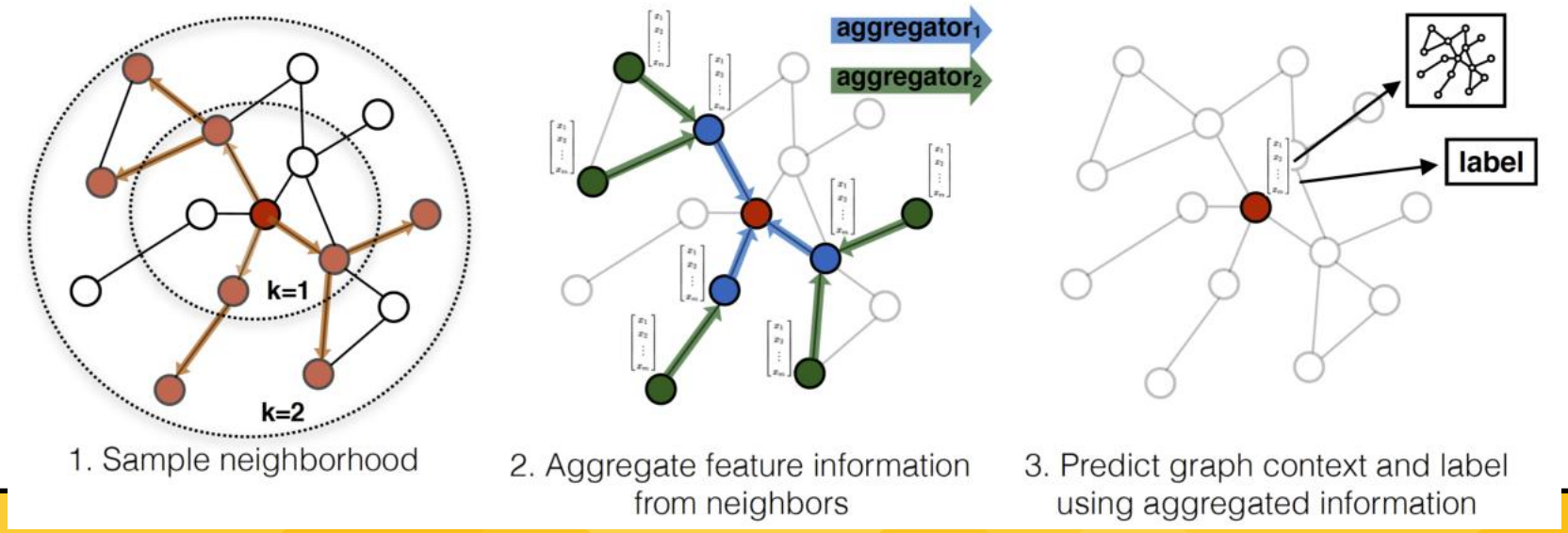
$$H^{(l+1)} = W_1 H^{(l)} + A H^{(l)} W_2$$

- Why? For the i-th row of $H^{(l+1)}$

$$h_i^{(l+1)} = W_1 h_i^{(l)} + \sum_j A_{ij}\, h_j^{(l)} W_2$$

GraphSAGE

- Inductive learning through sampling (Hamilton et al. 2017)
- Sample k random neighbors (not all) Aggregate their embeddings
- Learn inductive functions for new nodes
- Key advantage: Works on unseen graphs, scalable



GraphSAGE

Algorithm 1: GraphSAGE embedding generation (i.e., forward propagation) algorithm

Input : Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; input features $\{\mathbf{x}_v, \forall v \in \mathcal{V}\}$; depth K; weight matrices $\mathbf{W}^k, \forall k \in \{1, ..., K\}$; non-linearity σ; differentiable aggregator functions $\text{AGGREGATE}_k, \forall k \in \{1, ..., K\}$; neighborhood function $\mathcal{N} : v \rightarrow 2^{\mathcal{V}}$

Output : Vector representations $\mathbf{z}_v$ for all $v \in \mathcal{V}$

$\mathbf{h}_v^0 \leftarrow \mathbf{x}_v, \forall v \in \mathcal{V}$;

for $k = 1...K$ **do**

  for $v \in \mathcal{V}$ **do**

    $\mathbf{h}_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\})$;

    $\mathbf{h}_v^k \leftarrow \sigma\left(\mathbf{W}^k \cdot \text{CONCAT}(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^k)\right)$

  end

  $\mathbf{h}_v^k \leftarrow \mathbf{h}_v^k / \|\mathbf{h}_v^k\|_2, \forall v \in \mathcal{V}$

end

$\mathbf{z}_v \leftarrow \mathbf{h}_v^K, \forall v \in \mathcal{V}$

Over smooth and graph attention

Issues with GCNs

- The standard GCN uses a predefined, isotropic (uniform) aggregation function.
- This means that when a node receives messages from its neighbors, it treats all neighbors equally irrespective of their importance, feature relevance, or the nature of their connection.

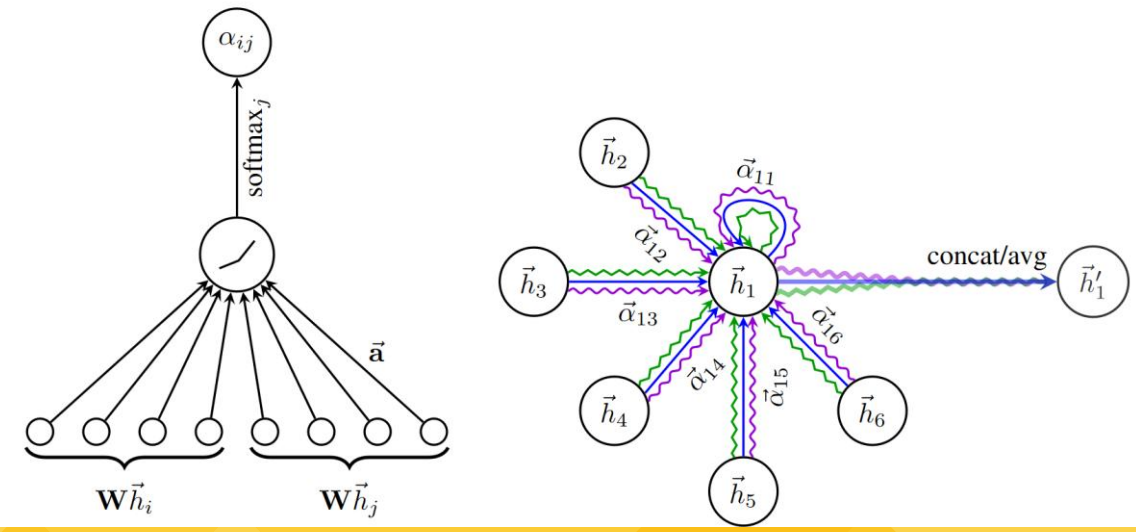
- Before aggregation, the current node features are transformed by a learnable weight matrix (shared by all nodes)

- The normalized adjacency matrix ($\hat{A}$) is the "convolution" part

Graph Attention Networks (GAT)

- Learn adaptive neighbor weights (Veličković et al. 2018)
- Attention mechanism:
- $\alpha = softmax(LeakyReLU(a[Wh_i || Wh_j]))$
- Each node learns which neighbors are important

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T[\mathbf{W}\vec{h}_i \| \mathbf{W}\vec{h}_j]\right)\right)}{\sum_{k\in\mathcal{N}_i}\exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T[\mathbf{W}\vec{h}_i \| \mathbf{W}\vec{h}_k]\right)\right)}$$

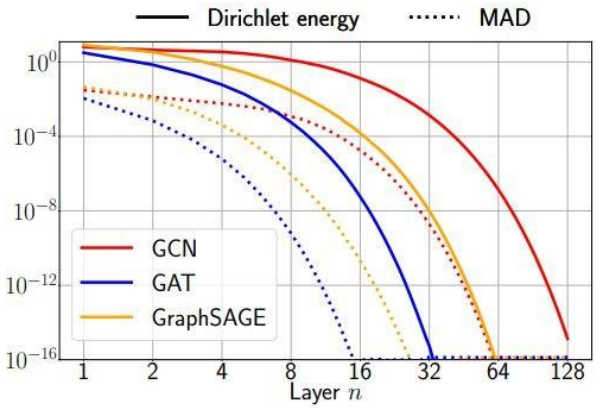


Training & Optimization

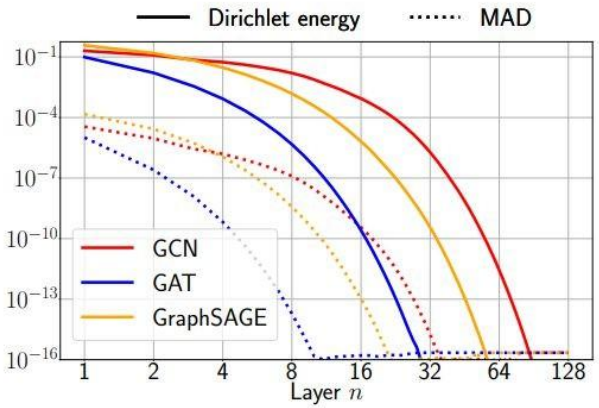
- Layer depth: 2-4 layers optimal (over-smoothing at depth)
- Pooling: Global sum/mean for graph-level tasks
- Batching: Mini-batch graphs or node sampling
- Regularization: Dropout, weight decay, batch norm

The Over-Smoothing Problem

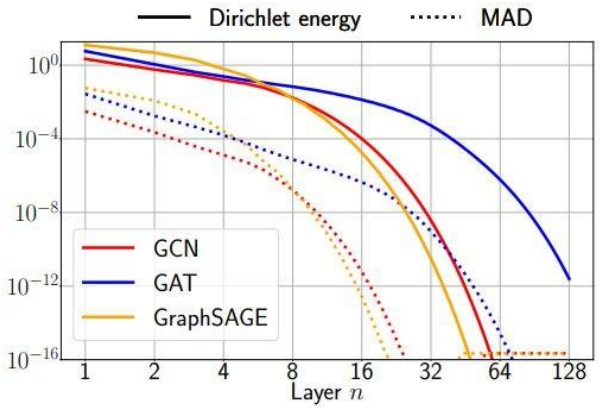
- As layers increase, node embeddings converge to similar values
- Problem:
  - Deep GNNs lose expressiveness. Nodes become indistinguishable.
  - This is why 2-4 layers is standard practice



Small-scale Texas graph



medium-scale Cora citation network

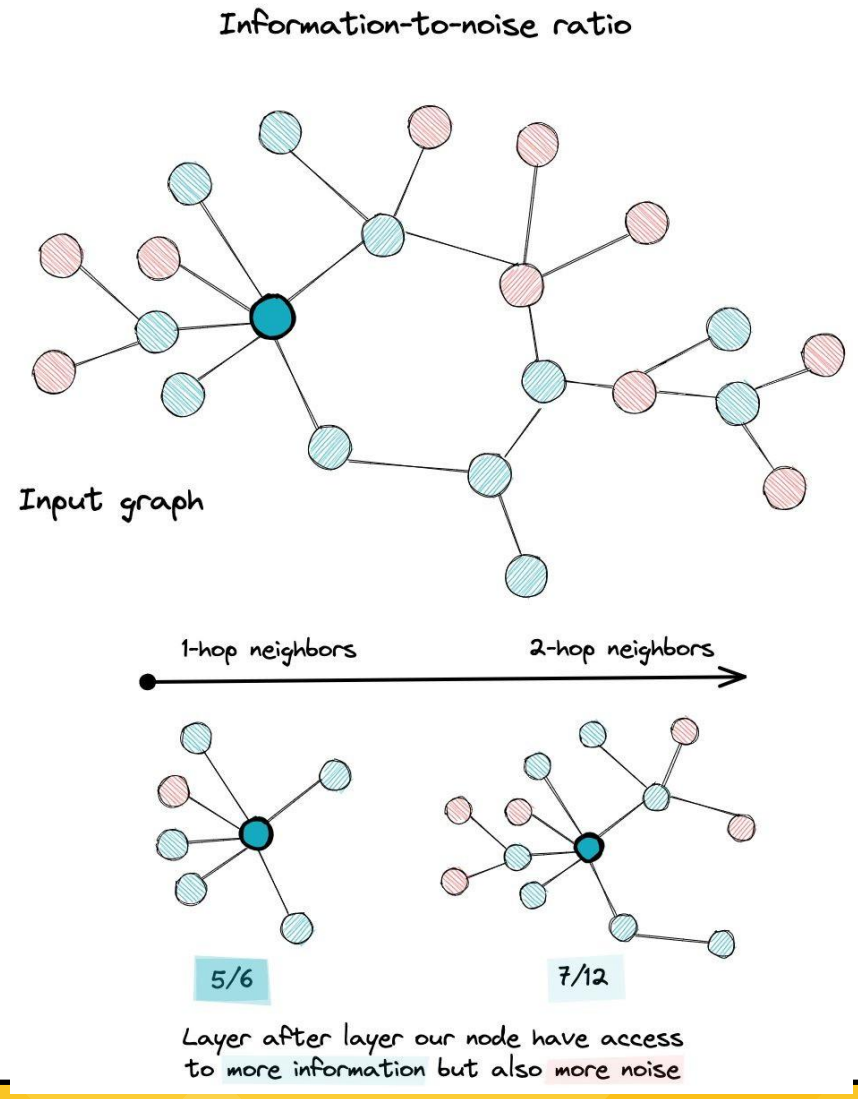


large-scale facebook network

The Over-Smoothing Problem

- This generalization was built on the main hypothesis that while nodes are interacting, they have either access to important information from nodes from the same class or noise by interacting with nodes from other classes.

- While node access to more parts of the graph we may access to noisy nodes that affect the final embedding.

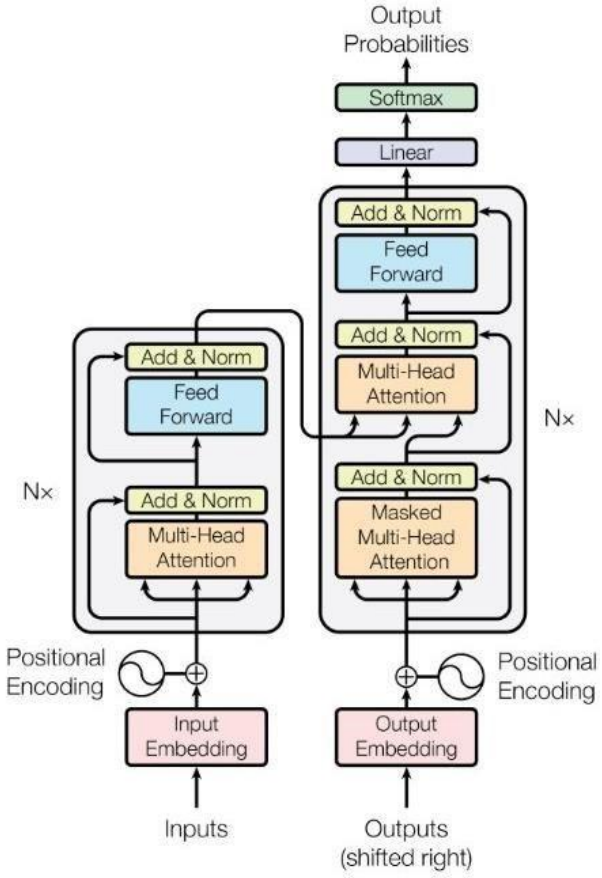


GAT's Limitation

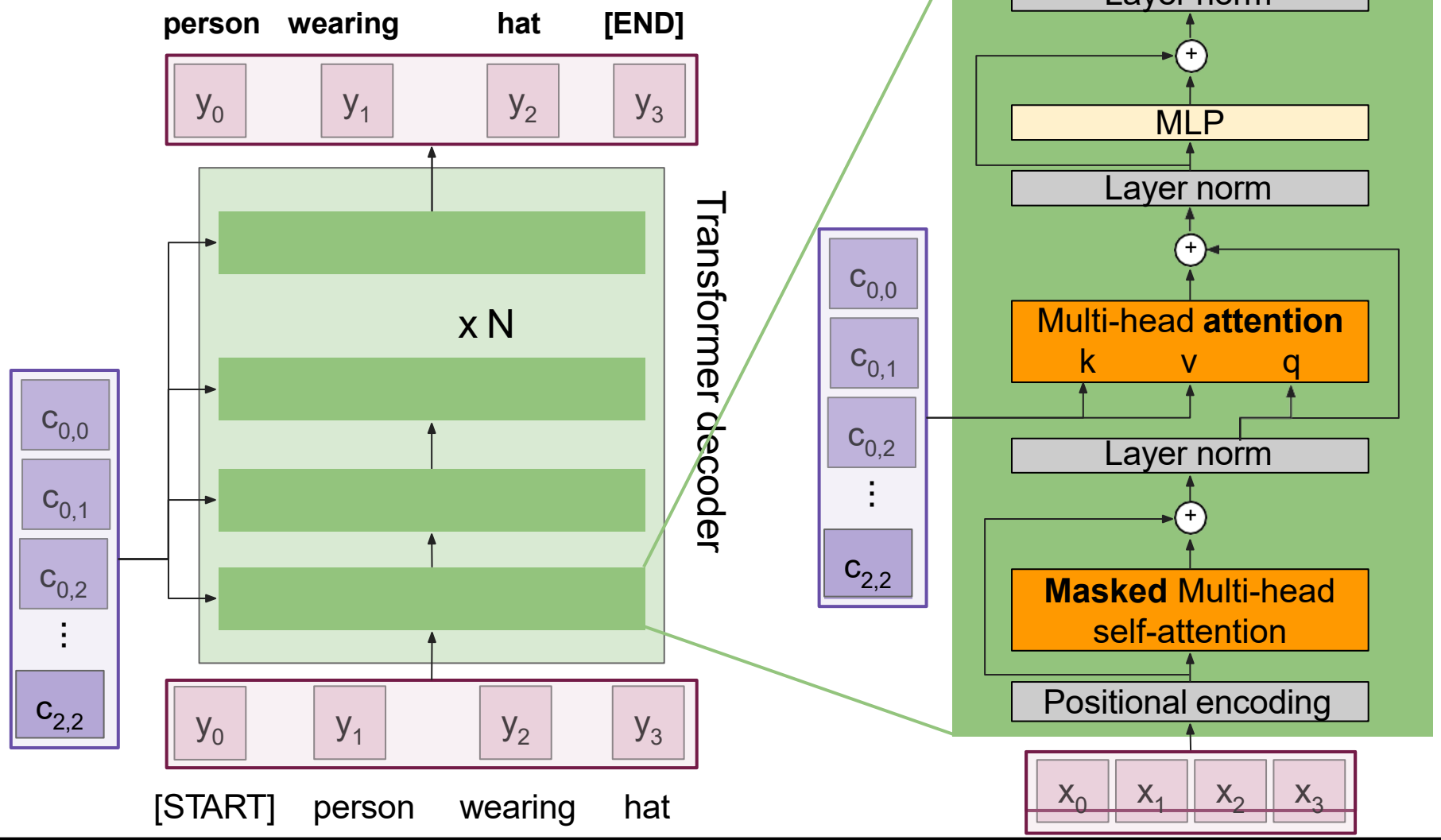
- What is Graph attention (GAT)? Like the GCN, the GAT still operates within the confines of the Message Passing paradigm i.e aggregates information from a node's immediate neighbors (1-hop)
- Calculates weights based on node features and local connectivity, it doesn't explicitly encode the full positional or structural relationship between two nodes that are far apart
- Attention is calculated only on a subset of features (neighbors), limiting the complexity of the feature interactions
- To capture global patterns or long-range dependencies, you must stack many layers (L).
- Leads to the over-smoothing problem

Recap Transformers

- Self-Attention is used to weigh the importance of all other words in the sequence when processing one word.
- Compute attention scores using a function of Q(Query), K (Key), and V (Value) matrices, typically via the Scaled Dot-Product Attention
- Use a Feed-Forward Network block after the attention mechanism to enhance expressiveness.



The Transformer Decoder block



Transformer Decoder Block:

Inputs: Set of vectors **x** and Set of context vectors **c.**
Outputs: Set of vectors **y.**

Masked Self-attention only interacts with past inputs.

Multi-head attention block is NOT self-attention. It attends over encoder outputs.

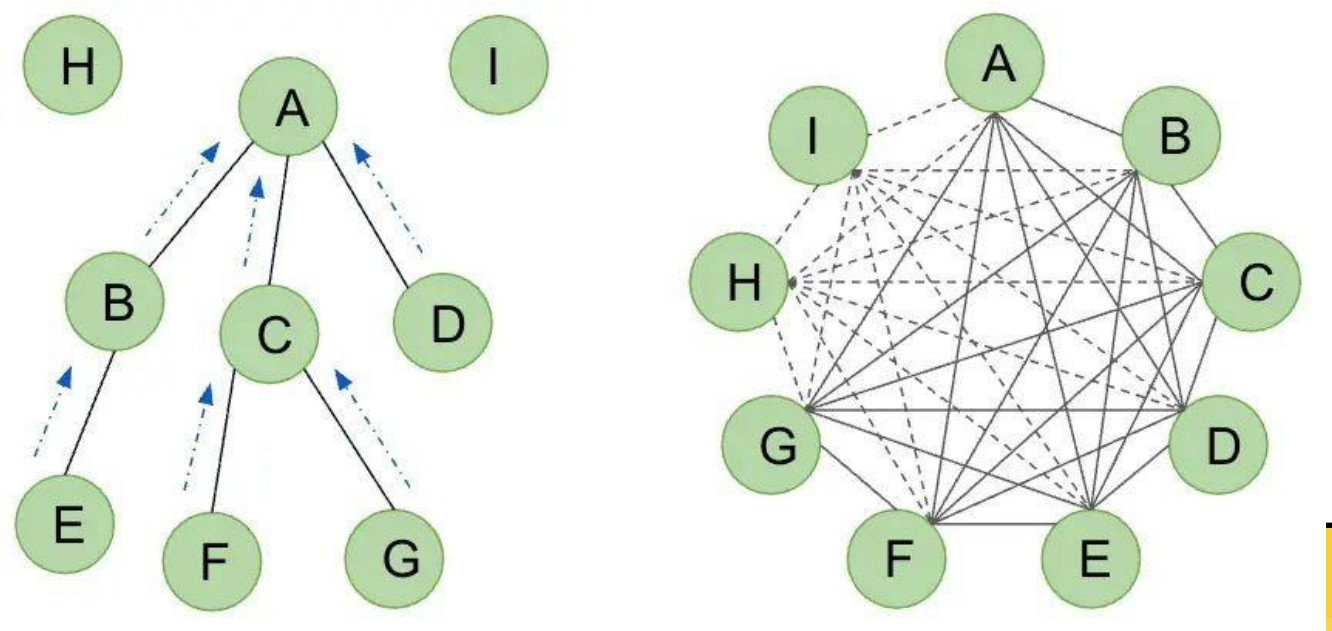
Highly scalable, highly parallelizable, but high memory usage.

Differences between Transformer (NLP) and Graph Transformer (GNN)

- The differences arise from the need to adapt the sequence-based operations of the original Transformer to the irregular, non-sequential structure of a graph
  - **Attention Scope**: Customizable for Local Attention (similar to GAT) or Global Attention
  - **Positional Encoding**: Cannot use sinusoidal function or learnable vector. No sequential order. Must incorporate Graph Structure Encoding
  - **Structural Encoding**: Explicitly models graph properties
  - **Masking**: Cannot use Causal Masking. Adjacency Masking is an option

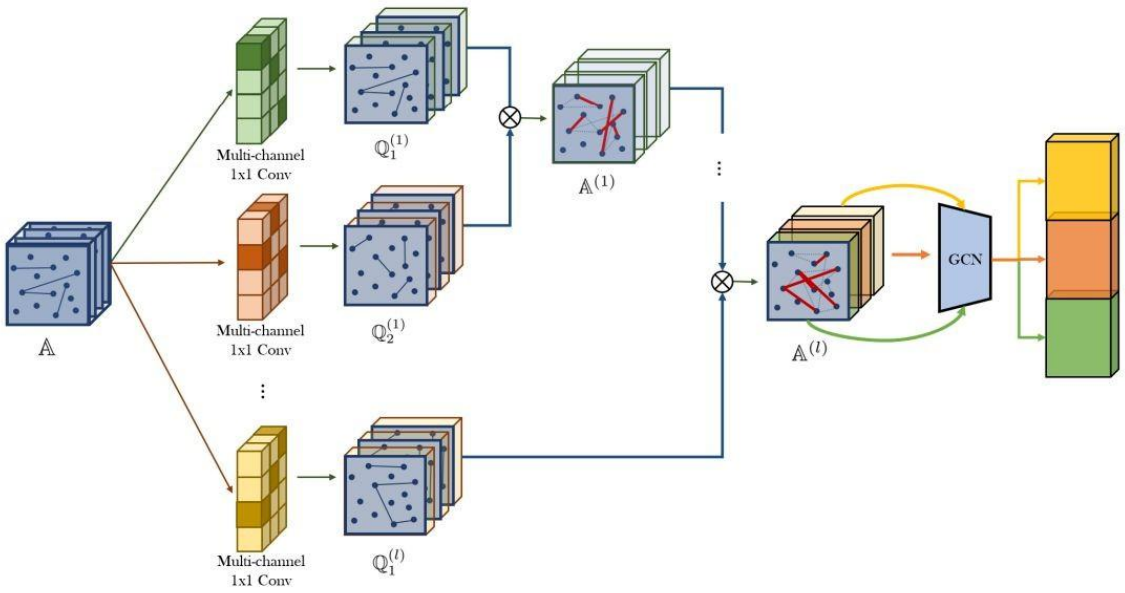
Graph Transformer (GNN) Information Flow

- Left: GNN with local message passing. Disconnected nodes like “H” and “I” do not influence “A.”
- Right: Graph Transformer with global attention. “A” can attend to all nodes, capturing long-range dependencies.



Graph Transformers

- Full attention over all nodes (no just neighbors)
- Spectral Transformers: Learnable PE (positional encoding)
- Graph-scaled attention: Efficiency via subgraph sampling
- SOTA on many benchmarks



Confused??

- Graph Neural Network (GNNs)
- Graph Convolutional Networks (GCNs)
- Graph Attention Networks (GAT)
- Graph Transformer (GT)

Summary

Architecture	Pros	Cons	When to Use
GNN	Message-passing paradigm.	Not a model	Parent of all Graph model!
GCN	Computationally fast	Treats all neighbors equally	For a quick, efficient solution on medium-sized, simple graphs
GAT	Assigns different, learned importance scores to neighbors using attention	High Compute Cost. Still limited to local aggregation. risk over smoothing in deep models	High-Relational Data (social networks or knowledge graphs)
Graph Transformer	Long-range dependencies, structural encoding No over-smoothing	Highest Compute Cost. Attention $O(N^2)$ for N nodes. Complicated	Long-range dependencies are critical. Prioritize expressiveness over speed. Eg (protein folding)

Ecosystems for GNNs

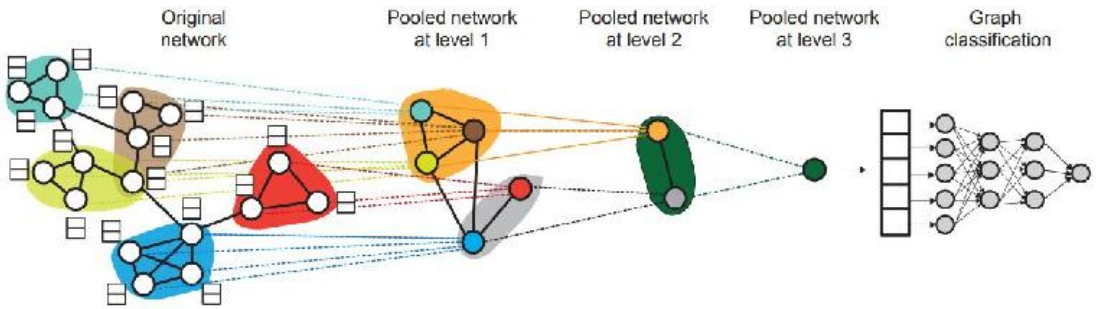
Ecosystems for GNNs

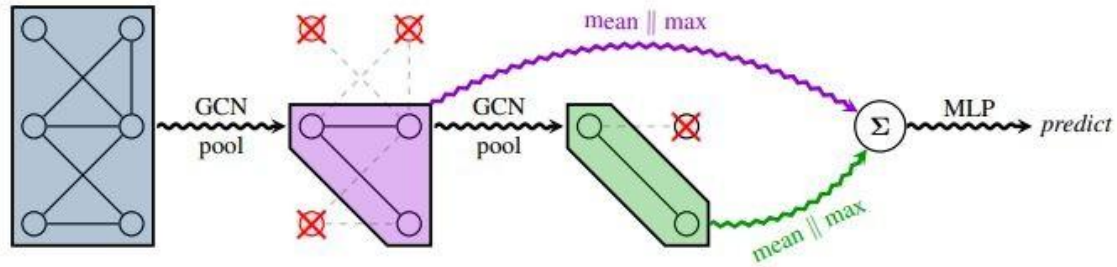
- PyTorch Geometric: Industry standard, rich models, production-ready
- DGL: Framework-agnostic, optimized for large graphs
- NetworkX: Data preprocessing, analysis, visualization

Advanced Topics

Hierarchical Graph Pooling

- Coarsen graphs like CNNs pool spatial dimension
- DiffPool: Differentiable pooling with assign
- TopK pooling: Select important nodes
- Benefit: Multi-scale graph representations

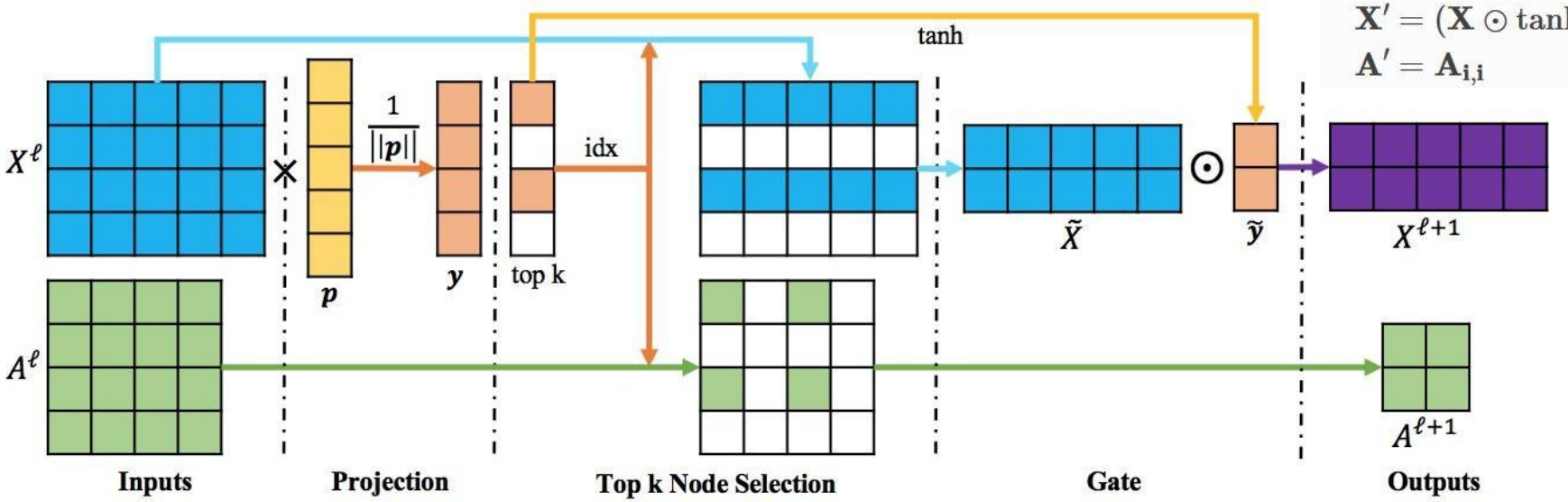




Top-k Pool (originally gpool)

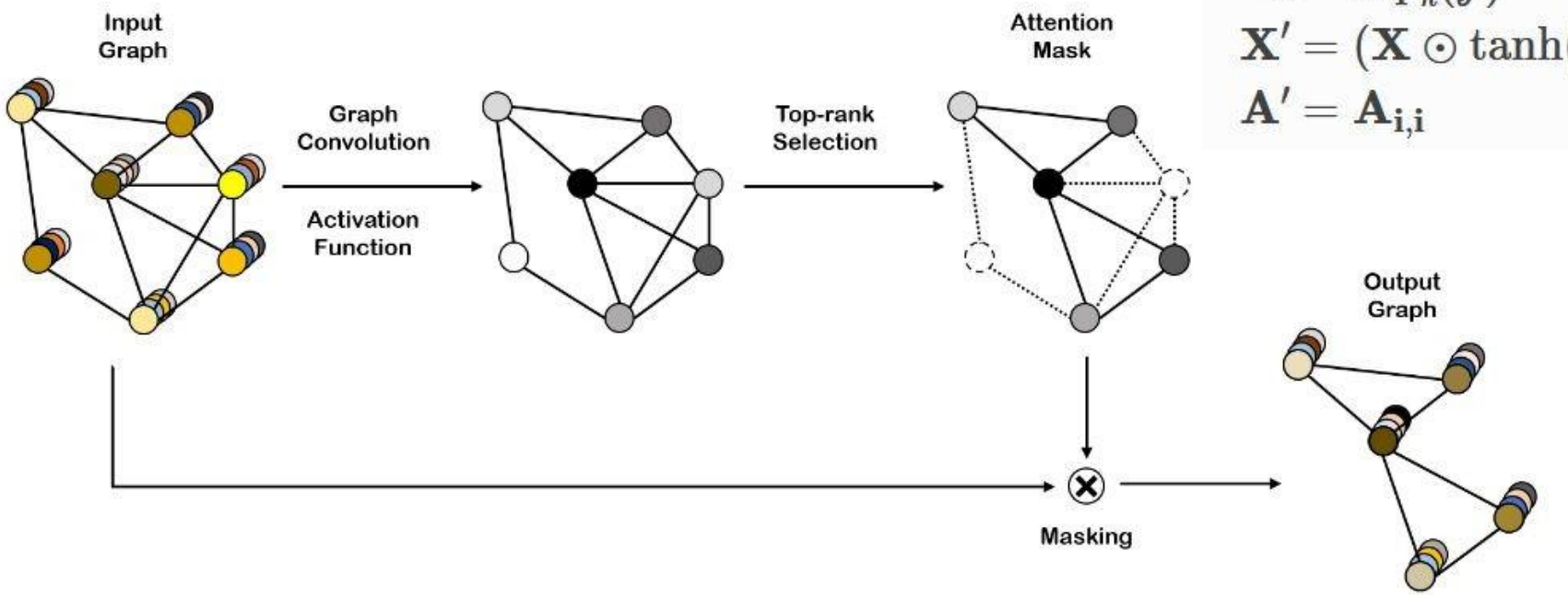
- Autoencoder with a trainable projection vector

$$\mathbf{y} = \sigma\left(\frac{\mathbf{X}\mathbf{p}}{\|\mathbf{p}\|}\right)$$
$$\mathbf{i} = \mathrm{top}_k(\mathbf{y})$$
$$\mathbf{X}' = (\mathbf{X} \odot \tanh(\mathbf{y}))_{\mathbf{i}}$$
$$\mathbf{A}' = \mathbf{A}_{\mathbf{i},\mathbf{i}}$$



Gao and Ji: Graph U-Net (ICLR 2019 submission) and Cangea et al.: Towards Sparse Hierarchical Graph Classifiers (NeurIPS-W 2018)

Self-attention Graph Pooling (sagpool)



$$\mathbf{y} = \mathrm{GNN}(\mathbf{X}, \mathbf{A})$$
$$\mathbf{i} = \mathrm{top}_k(\mathbf{y})$$
$$\mathbf{X}' = (\mathbf{X} \odot \tanh(\mathbf{y}))_{\mathbf{i}}$$
$$\mathbf{A}' = \mathbf{A}_{\mathbf{i},\mathbf{i}}$$

Lee, Lee, Kang: Self-Attention Graph Pooling (ICML 2019)

Main Idea: How do we represent images using graphs?

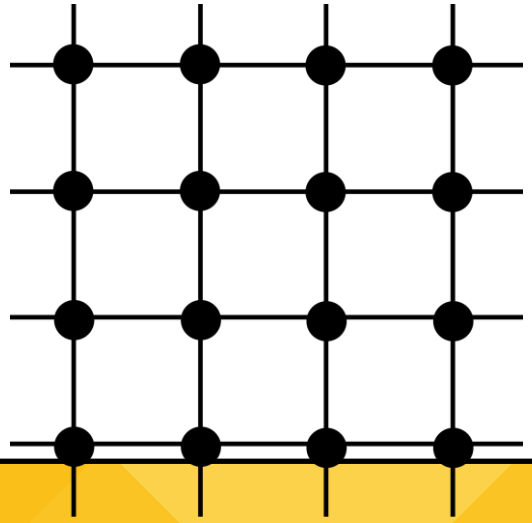
MNIST

28 x 28 pixel images

Grid Graph

28 * 28 = 764 nodes

764 x 764 adj



Graph CNNs (GCN, GAT, ...)

<u>Most</u> <u>intuitive:</u>

Nodes = pixels,

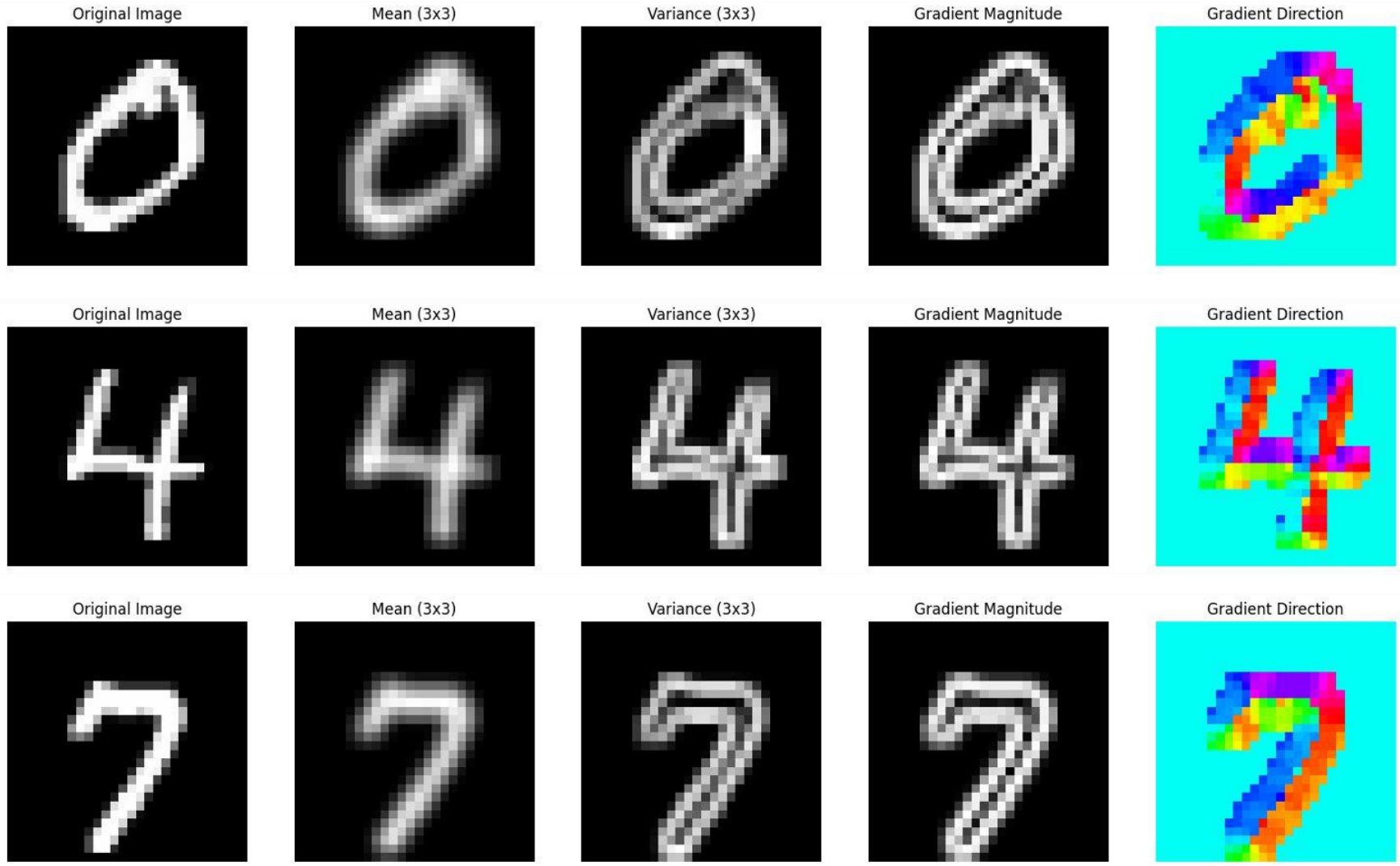
Edges connect Adjacent Pixels,

Features = pixel intensities

What about the features?

- 1 Feature: Pixel values.
- Can we use more descriptive features/graph signals besides pixel values?
  - Mean at each pixel position
  - Variance at each pixel position
  - Gradient Magnitude
  - Gradient Direction

Example Features



Features also affect accuracy!

GNN Model	Number of Node Features	Test Accuracy (mean_acc ± std_deviation)
GCN	1 (Pixel Intensity)	0.5505 ± 0.0289
GCN	4 (Mean, Variance, Gradient Magnitude, Gradient Direction)	0.7206 ± 0.0401
GAT	4 (Mean, Variance, Gradient Magnitude, Gradient Direction)	0.7818 ± 0.0006

Summary

- Introduction to GNN and Challenges
- Graph basics
- Graph neural networks
- Over smooth and graph attention
- Ecosystems for GNNs
- Advanced topics